Active Reader: A Hierarchical Knowledge-Driven System for Adaptive Educational Content Generation

Technical Architecture Document *Foundations of Computer Vision — Active Reading Platform*

Abstract

This document describes the architecture of the Active Reader system, a two-stage pipeline for transforming static textbook content into adaptive, interactive learning experiences. Stage 1 constructs a richly annotated concept graph from raw book text using a multi-pass LLM extraction pipeline. Stage 2 compiles each concept node into a hierarchical lesson state machine and tracks student progress through a persistent learner model. The system draws on established patterns from hierarchical finite state machines (HSMs) in game engine design, spaced repetition scheduling, and knowledge graph pedagogy (Metacademy, DeepTutor) to produce three output modalities: interactive book reading, slide generation, and structured video scripting.

1. Introduction

Textbooks are static artifacts. They encode expert knowledge linearly — chapter by chapter — with no awareness of a reader's prior knowledge, current struggles, or preferred modality of learning. The Active Reader system addresses this gap by treating a textbook not as a sequence of pages but as a **concept graph**: a directed acyclic graph (DAG) in which nodes are atomic knowledge units and edges encode prerequisite relationships.

The system is designed around a two-stage architecture:

- **Stage 1 (Complete):** Parse the raw textbook into a concept graph where each node carries rich semantic metadata — verbatim passages, motivations, key moments, exercises, figures, and provenance annotations.
- **Stage 2 (Proposed):** Compile each concept node into a hierarchical lesson plan, execute it through a state machine runtime, and adapt delivery based on a persistent student model.

The core insight is that the concept graph is not merely a navigation aid — it is the **complete specification** from which all downstream lesson content can be derived deterministically.

2. Stage 1: Concept Graph Construction

2.1 Pipeline Overview

The extraction pipeline processes a raw textbook (QMD, Markdown, or PDF) through 21 sequential stages, orchestrated by a fault-tolerant runner with per-stage checkpointing:

Stage Group	Stages	Purpose
Ingestion	1–5	Parse structure, download images, enrich markdown, number lines
Chunking	6–7	Segment book into ~2000-token chunks; audit chunk boundaries
Extraction	8–11	LLM pass extracts concepts and items per chunk; verify; salvage
Normalization	12–16	Splice, dedup, format concepts and items; link items to concepts
Edge Extraction	17–18	Extract prerequisite and overlay edges; validate
Slot Filling	19–21	Fill key_passage, motivation, question, example per concept

The primary LLM for extraction (Pass A) is `gemini-2.5-flash`; formatting and edge extraction use `gpt-4.1-mini`. All LLM calls are wrapped with exponential backoff and rate-limit-aware retry (30s base wait on 429 errors) with worker counts tuned to avoid TPM exhaustion.

2.2 Concept Node Schema

Each node in the graph is a richly annotated record:

```
Concept {
  // Identity
  id:           slug identifier
  kind:         definition | theorem | technique | idea
  title:        human-readable name
  aliases:      alternative names

  // Content (book-grounded)
  content:      paraphrased book text (same claims, clean prose)
  key_passage:  verbatim book quote (most important sentence)
  motivation:   verbatim book sentence explaining why this matters
  recap_md:     bullet cheat-sheet
  question:     study question (verbatim when present in book)
  example:      worked example text

  // Links
  item_ids:     { figures, examples, exercises, theorems, tables }
  tags:         keyword labels

  // Position
  position:     { chapter, section, book_order, first_line }
  source:       { file, spans: [{start: L00001, end: L00010}] }

  // Meta
  _provenance:  per-field source (book_extracted | book_paraphrased | llm_inferred)
  quality_flags: non-blocking warnings
}
```

2.3 Current Graph Statistics (Foundations of Computer Vision)

Metric	Value
Total concepts	1,267
Chapters	55
Prerequisite edges	~2,400
Concepts with key_passage	98% (1,241)
Concepts with motivation	71% (902)
Concepts with question	10% (131)
Linked figures	871 images
Linked exercises	~800 items

---

3. Stage 2: Lesson State Machine Architecture

3.1 Design Principles from Game Engine Architecture

The lesson execution system is modelled on hierarchical finite state machines (HSMs) as used in game engines. Three principles are adopted:

Separation of concerns. A game engine separates the *game loop* (scheduler), *level data* (static content), *player state machine* (per-entity logic), and *render loop* (output projection). Correspondingly, Active Reader separates the *session scheduler*, *lesson plan* (compiled static JSON), *lesson state machine* (runtime executor), and *renderer* (slides / video / text).

Compound states with lifecycle methods. Each state defines `enter()` (fires once on entry, triggers rendering), `update()` (handles events), and `exit()` (fires once on departure, writes to student model). Child states inherit parent transitions, ensuring that global events (close app, jump to map) always terminate cleanly regardless of depth.

State stack for layered contexts. A push/pop stack allows overlay states (hint panels, question dialogs, pause) to sit on top of the running lesson without destroying it — analogous to a pause menu in a game.

3.2 Content Hierarchy

```
BOOK  (compound)
└── CHAPTER  (compound, 1..55)
    └── CONCEPT  (compound, 1..N)
        └── LESSON  (compound)
            ├── HOOK
            ├── MOTIVATE
            ├── EXPLAIN
            ├── VISUAL
            ├── KEY_MOMENT
            ├── EXAMPLE
            ├── PRACTICE
            │   ├── ATTEMPT
            │   ├── HINT
            │   └── RETRY
            └── RECAP
```

Each level owns transitions that apply to all its children. At the BOOK level: save-on-exit, restore-cursor-on-entry. At the CHAPTER level: set chapter context, expose cross-concept references, unlock chapter review on completion. At the CONCEPT level: verify prerequisites, load concept node data.

3.3 Lesson Plan as Static Compiled Data

The lesson plan is a JSON document compiled deterministically from the concept node — it is not generated at runtime. The compiler maps concept fields to lesson segments:

```
concept.question        →  HOOK segment
concept.motivation.text →  MOTIVATE segment
concept.content         →  EXPLAIN segment
concept.item_ids.figures→  VISUAL segment (one per figure)
concept.key_passage     →  KEY_MOMENT segment
concept.item_ids.examples → EXAMPLE segment
concept.item_ids.exercises → PRACTICE segment
concept.recap_md        →  RECAP segment
```

When a required field is absent (e.g. no `question`), the compiler calls a single cheap LLM pass (`gpt-4.1-mini`) to generate a substitute. This is the only LLM call at Stage 2 compile time — all other content is derived from Stage 1 outputs.

3.4 Lesson State Machine (Runtime)

```
LessonMachine {
  initial: 'hook',
  states: {
    hook:      { enter: render(plan.hook),      on: { CONTINUE: 'motivate', SKIP: 'explain' }},
    motivate:  { enter: render(plan.motivate),  on: { CONTINUE: 'explain',  BACK: 'hook' }},
    explain:   { enter: render(plan.explain),   on: { CONTINUE: next_segment(),
                                                      QUESTION: 'asking',
                                                      BACK: 'motivate' }},
    visual:    { enter: render(plan.visual),    on: { CONTINUE: 'example' }},
    key_moment:{ enter: render(plan.key_moment),on: { CONTINUE: 'example' }},
    example:   { enter: render(plan.example),   on: { CONTINUE: 'practice', BACK: 'explain' }},
    practice:  {
      initial: 'attempt',
      states: {
        attempt: { on: { CORRECT: '#recap', WRONG: 'feedback' }},
        feedback:{ on: { RETRY: 'attempt', HINT: 'hint' }},
        hint:    { on: { RETRY: 'attempt', GIVE_UP: '#explain' }},
      }
    },
    recap:     { enter: render(plan.recap),     on: { DONE: 'complete' }},
    asking:    { enter: open_question_dialog(), on: { ANSWERED: 'explain', DISMISS: 'explain' }},
    complete:  { type: 'final', enter: write_to_student_model() }
  }
}
```

The renderer is fully decoupled — the same state machine drives slides (one state = one slide), video (one state = one narration segment + visual), and book reading (linear text with interactive checkpoints).

3.5 Chapter-Level State

```
ChapterMachine {
  initial: 'intro',
  states: {
    intro:   { enter: render_chapter_overview(),  on: { BEGIN: 'concepts' }},
    concepts:{
      type: 'compound',
      // iterates through ordered concept list
      onDone: 'review'   // fires when all child concepts reach 'complete'
    },
    review:  { enter: render_synthesis_exercise(), on: { PASS: 'complete', FAIL: 'concepts' }},
    complete:{ type: 'final' }
  }
}
```

3.6 Adaptive Transitions

Transitions adapt based on student model state at runtime. Examples:

Condition	Adaptation
`retention_score < 0.4` on a prereq	Insert brief prereq recap before EXPLAIN
`attempts > 3` on practice	Surface HINT automatically
`prefers_visual = true`	Prioritise VISUAL segment, expand figures
`skips_recap = true` (inferred)	Make RECAP collapsible, auto-advance
`concept in struggle_profile.weak_tags`	Slow down, add extra example

---

4. Student Model

The student model is a persistent record orthogonal to the content hierarchy. It is the "save file" — written after every interaction and read by the session scheduler to determine what to present next.

4.1 Schema

```
StudentModel {
  cursor: {
    last_position:   concept_id + lesson_state,
    last_active:     timestamp,
    session_count:   int
  }

  concept_records: {
    [concept_id]: {
      status:          LOCKED | SEEN | PRACTICED | MASTERED,
      attempts:        int,
      last_correct:    timestamp,
      retention_score: float,   // decays via forgetting curve
      time_spent_ms:   int
    }
  }

  struggle_profile: {
    weak_tags:         string[],    // tags of low-retention concepts
    stuck_concepts:    string[],    // failed > N times
    skipped_concepts:  string[],
    avg_attempts_by_kind: { definition, theorem, technique, idea }
  }

  interactions: [
    { type: question_asked | annotation | hint_requested |
            figure_expanded | answer_submitted,
      concept_id, content, timestamp }
  ]

  preferences: {
    prefers_visual:      bool,
    skips_recap:         bool,
    avg_reading_speed:   words_per_min
  }
}
```

4.2 Spaced Repetition Scheduler

The session scheduler functions as the game loop — it runs at session start and after each concept completion to determine what to present next:

```
scheduler(student_model, concept_graph) → next_concept_id

algorithm:
  1. collect all AVAILABLE concepts (prereqs met, not MASTERED)
  2. score each by: due_date (spaced repetition) × difficulty × weakness_bonus
  3. return highest score
  4. on session start: surface any concepts where retention_score < threshold (review queue)
```

The forgetting curve decays `retention_score` over time since `last_correct`, modulated by the number of successful recalls. This is the "lookup table indexed by timer" from the game loop analogy — the student model is the RAM, the scheduler is the game loop reading from it.

---

5. Output Compilation

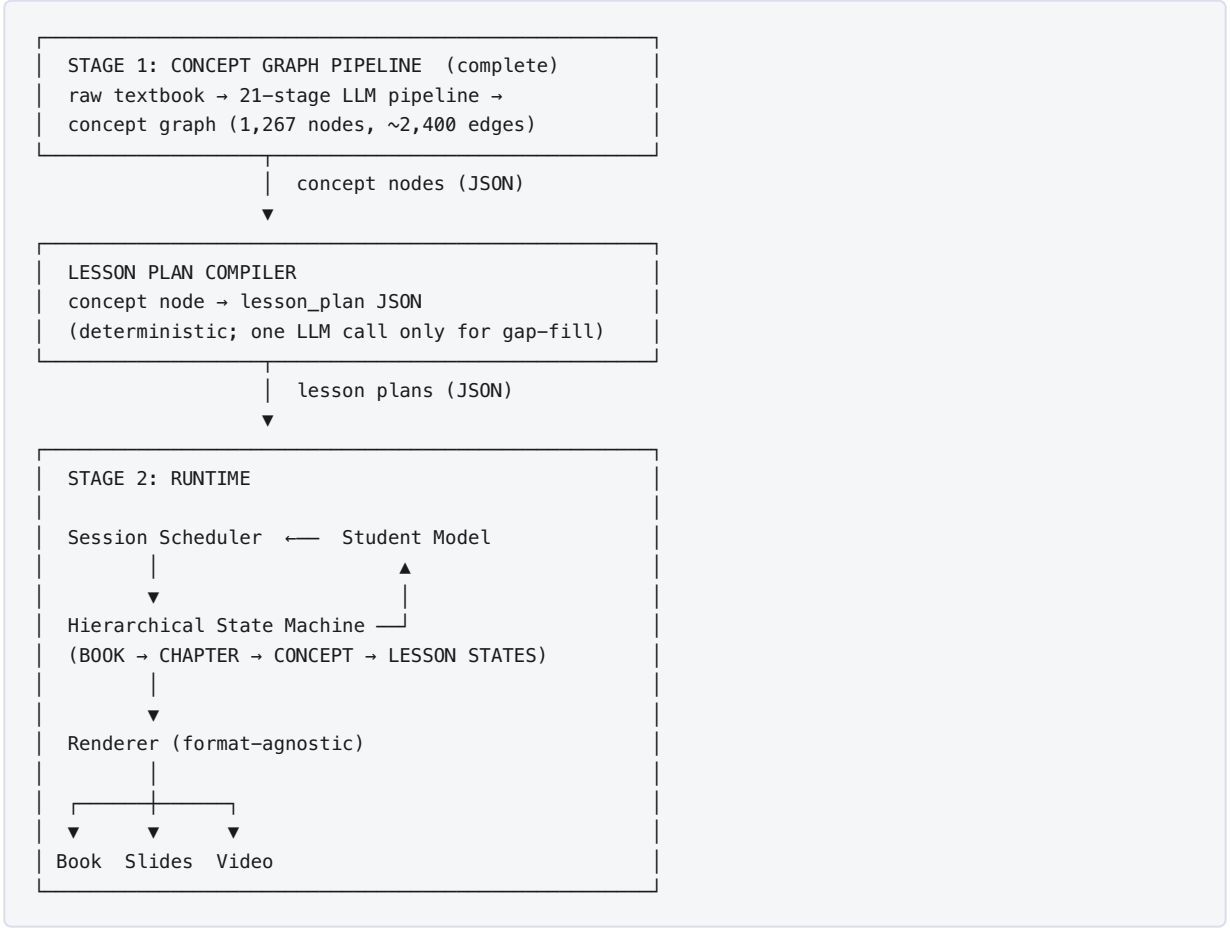
The same lesson plan and state machine compile to three output formats:

Format	How state maps to output
Interactive Book Reading	States rendered inline as collapsible panels; student progresses by scrolling/clicking
Slides	Each top-level state = one slide; figures and key_passage get full-bleed treatment
Video Script	Each state = narration segment + visual directive; compiled to a timed script for TTS/ recording

The renderer is the only component that differs between formats. The lesson plan, state machine, and student model are format-agnostic.

---

6. Summary Architecture



---

7. Next Steps

Priority	Task
1	Define lesson plan JSON schema and write compiler (concept node → lesson_plan.json)
2	Implement lesson state machine using XState or custom HSM
3	Define student model schema and persistence layer
4	Build interactive book reader renderer (simplest output format)
5	Implement session scheduler (spaced repetition)
6	Add slides and video script renderers
7	Re-run pipeline on full vision book with rate-limit-safe settings